



**Universidad Tecnológica Nacional
Facultad Regional Córdoba**

Algoritmos y Estructuras de Datos

Ficha Nro. 14

Arreglos: Caso de Estudio I

Prof. Ing. Corso Cynthia

Temario

- Repaso de ficha 12 y 13.
- Arreglos correspondientes o Paralelos.
- Arreglos de conteo y/o acumulación.

Arreglos correspondientes o paralelos

- Los arreglos “correspondientes” o “paralelos” se utilizan cuando es necesario almacenar información en varios arreglos unidimensionales a la vez. De tal forma que exista una **correspondencia entre los valores almacenados en las casillas con el mismo índice (llamadas casillas homólogas)**.
- Por ejemplo: puede ser que sea necesario guardar en un arreglo los nombres de ciertas personas y en otro el importe del sueldo que perciben.

<i>nombres</i>	Juan	Ana	Alejandro	María	Pedro
<i>sueldos</i>	1100	2100	1300	750	800
	0	1	2	3	4



Ambos arreglos deben tener la misma cantidad de componentes

Acceso y visualización: arreglos paralelos

- El manejo de ambos arreglos es simple: solo se debe recordar que hay que usar **el mismo índice en ambos arreglos** para acceder a la información de la misma persona. Siguiendo este esquema, visualiza por pantalla el nombre y el sueldo de la persona en la casilla 2.
- Ejemplo: asumiendo que los arreglos ya están cargados:

```
print('Nombre:', nombres[2]) # Alejandro  
print('Sueldo:', sueldos[2]) #1300
```



<i>nombres</i>	Juan	Ana	Alejandro	María	Pedro
<i>sueldos</i>	1100	2100	1300	750	800
	0	1	2	3	4

Carga y visualización de arreglos paralelos

- El primer script permite la carga de los dos arreglos paralelos. El primer arreglo contiene el nombre, mientras que en el segundo arreglo almacena el sueldo que percibe el empleado.

```
def read(nombres, sueldos):  
    n = len(nombres)  
    for i in range(n):  
        nombres[i] = input('Ingrese nombre: ')  
        sueldos[i] = int(input('Ingrese sueldo: '))
```

- El segundo script visualiza los datos contenidos en los arreglos.

```
def display(nombres, sueldos):  
    n = len(nombres)  
    print('Datos de los empleados:')  
    for i in range(n):  
        print('Nombre:', nombres[i], '- Sueldo:', sueldos[i])
```

Script completo: arreglos paralelos

```
def validate(inf):
    n = inf
    while n <= inf:
        n = int(input('Cant. elementos (mayor a ' + str(inf) + ' por favor): '))
        if n <= inf:
            print('Se pidio mayor a', inf, '... cargue de nuevo...')
    return n

def test():
    n = validate(0)
    nombres = n * [' ']
    sueldos = n * [0]
    print('\nCargue los datos de las personas:')
    read(nombres, sueldos)
    display(nombres, sueldos)

# script principal...
if __name__ == '__main__':
    test()
```

Caso de Estudio: Arreglos paralelos

- Desarrollar un programa que permita cargar tres arreglos con n nombres de personas, sus edades y los sueldos que ganan. Luego de realizar la carga de todos los datos. Se solicita:
 - Informar el nombre de todos los empleados mayores a 18 años que cobren un sueldo mayor a 10.000 ordenados alfabéticamente.

<i>nombres</i>	Juan	Ana	Alejandro	María	Pedro
<i>edades</i>	30	18	25	43	38
<i>sueldos</i>	8000	15000	9000	40000	16000
	0	1	2	3	4

Caso de Estudio: Arreglos paralelos

- Para resolver este caso de estudio necesitamos implementar tres funciones: **i)** una que cargue los arreglos, **ii)** otra que ordene los elementos del arreglo, **iii)** y otra que muestre el listado solicitado.
- El script siguiente muestra la implementación de la primera función.

```
def read(nombres, edades, sueldos):  
    n = len(nombres)  
    for i in range(n):  
        nombres[i] = input('Nombre[' + str(i) + ']: ')  
        edades[i] = int(input('Edad: '))  
        sueldos[i] = int(input('Sueldo: '))  
    print()
```



<i>nombres</i>	Juan	Ana	Alejandro	María	Pedro
<i>edades</i>	30	18	25	43	38
<i>sueldos</i>	8000	15000	9000	40000	16000
	0	1	2	3	4

Caso de Estudio: Arreglos paralelos

- A continuación se muestran las funciones: **ii)** que ordena los elementos del arreglo (**Selección directa**), **iii)** y otra que muestre el listado solicitado.

```
def sort(nombres, edades, sueldos):  
    n = len(nombres)  
    for i in range(n-1):  
        for j in range(i+1, n):  
            if nombres[i] > nombres[j]:  
                nombres[i], nombres[j] = nombres[j], nombres[i]  
                edades[i], edades[j] = edades[j], edades[i]  
                sueldos[i], sueldos[j] = sueldos[j], sueldos[i]
```

```
def display(nombres, edades, sueldos):  
    n = len(nombres)  
    print('Mayores de 18 que ganan menos de 10000 pesos:')  
    for i in range(n):  
        if edades[i] > 18 and sueldos[i] < 10000:  
            print('Nombre:', nombres[i], '- Edad:', edades[i], '- Sueldo:', sueldos[i])
```

Caso de Estudio: Arreglos paralelos

- El siguiente es el script principal que activa las funciones anteriormente implementadas.

```
def validate(inf):  
    n = inf  
    while n <= inf:  
        n = int(input('Cant. de elementos (mayor a ' + str(inf) + ' por favor): '))  
        if n <= inf:  
            print('Error: se pidio mayor a', inf, '... cargue de nuevo...')  
    return n  
  
def test():  
    n = validate(0)  
    nombres = n * [' ']  
    edades = n * [0]  
    sueldos = n * [0]  
    print('\nCargue los datos de las personas:')  
    read(nombres, edades, sueldos)  
    sort(nombres, edades, sueldos)  
    display(nombres, edades, sueldos)  
  
if __name__ == '__main__':  
    test()
```

Vectores de conteo y de acumulación

- Los arreglos son estructuras de datos muy útiles, porque facilitan el **acceso directo a un componente**.
- En algunos casos esta característica facilita la resolución de problemas de manera simple y rápida. Por ejemplo: búsqueda binaria.
- Un **arreglo o vector de conteo** representa un arreglo en donde cada uno de sus **componentes** se comporta con un **contador**.
- Mientras que un **arreglo o vector de acumulación** cada **componente** se comporta como un **acumulador**.

Forma de vector de conteo

`vec[indice] = vec[indice] + 1`

Forma de vector de acumulación

`vec[indice] = vec[indice] + var`

Caso de Estudio: Vectores de conteo

- Cargar por teclado un conjunto de valores tales que todos ellos estén comprendidos entre 0 y 99 (incluidos ambos). Se indica el fin de datos con el número -1. Determinar cuantas veces apareció cada número.



Caso de Estudio: Vectores de conteo

Estrategia intuitiva de resolución:

- Usar un **ciclo** para cargar de a una vez un valor (*num*) por vez.
- El **ciclo se detiene** cuando el usuario ingrese el **valor de *num* igual a -1**.
- Usar un contador distinto, para determinar cuantas veces apareció cada valor posible de *num* pueda asumir. Por ejemplo: para contar cuantas veces apareció el **número 1**, se podría usar el contador **c1**, **para el 2** el contador **c2**, y así sucesivamente.
- Al finalizar el ciclo, se visualizan todos los contadores.

Caso de Estudio: Vectores de conteo

Limitaciones de la estrategia de resolución:

- Se deberían declarar e inicializar 100 contadores
- Luego dentro del ciclo, usar 100 condiciones anidadas!
- Además implica un gran esfuerzo de codificación y sobre todo la redundancia de instrucciones semejantes.



Caso de Estudio: Vectores de conteo

Una estrategia de resolución “eficiente”:

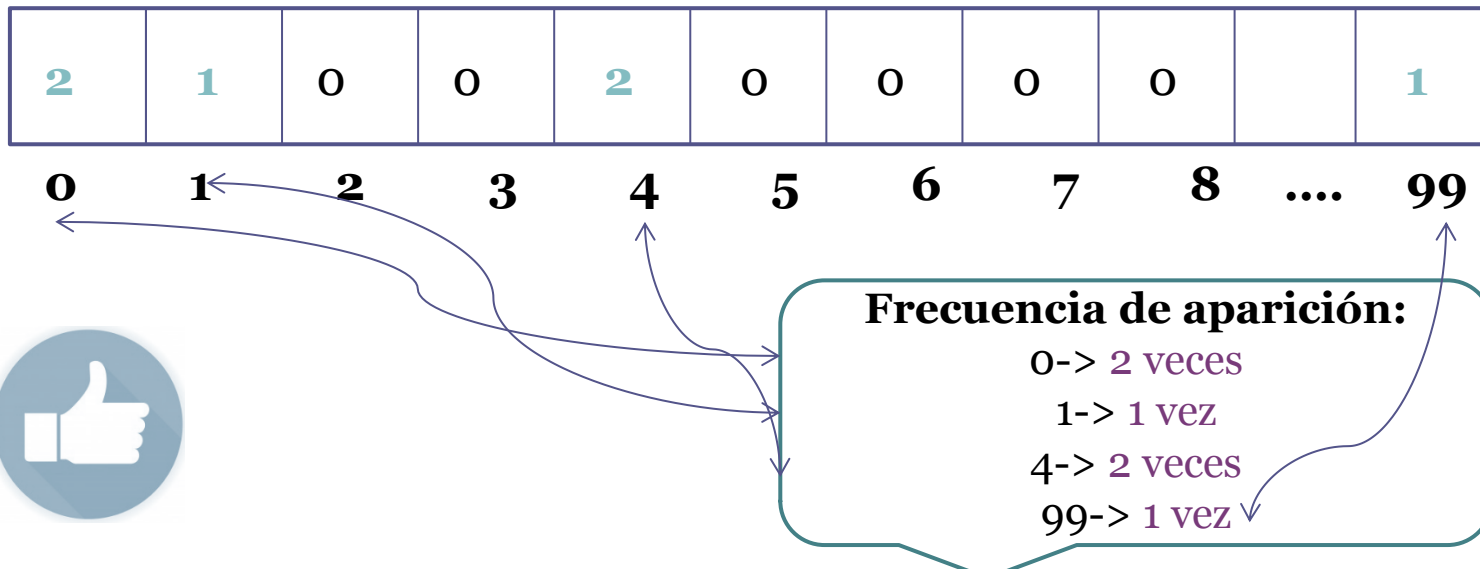
- Usar un arreglo unidimensional “*c*” de 100 componentes (vector de conteo), de forma que cada componente se use como uno de los contadores que se están necesitando.
- Se coloca inicialmente en cero cada componente del vector (dado que cada componente será un contador).
- Se usa un ciclo para cargar los valores *num*.
- Considerar el uso de **acceso directo** del arreglo unidimensional “*c*” para efectuar el conteo de cada numero ingresado.



Caso de Estudio: Vectores de conteo

Una posible estrategia de resolución:

- La idea central es que si *num* vale 0 (cero), entonces se debe usar $c[0]$ para contarlo, si *num* vale 1 (uno) usar $c[1]$
- En general para cada valor de la variable *num*, tal que:
 $0 \leq \text{num} \leq 99$, debe usarse $c[\text{num}]$ para contarlo.
- Ejemplo: Para la secuencia 0, 4, 0, 4, 1, 99, -1



Caso de Estudio: Vectores de conteo

Implementación del caso de estudio usando arreglos de conteo:

```
def test():
    n = 100
    c = n * [0]

    num = int(input('Ingrese un valor entre 0 y 99 (con -1 corta): '))
    while num != -1:
        if 0 <= num < n:
            c[num] += 1
        else:
            print('Error... el número debía ser >= 0 y <', n)
            num = int(input('Ingrese otro valor entre 0 y 99 (con -1 corta): '))

    print('Resultados:')
    for i in range(n):
        if c[i] != 0:
            print('Número', i, '- Frecuencia de aparición', c[i])

if __name__ == '__main__':
    test()
```

Muchas Gracias por la atención!!

Cualquier duda pueden contactarme por mensajería interna del Aula Virtual.